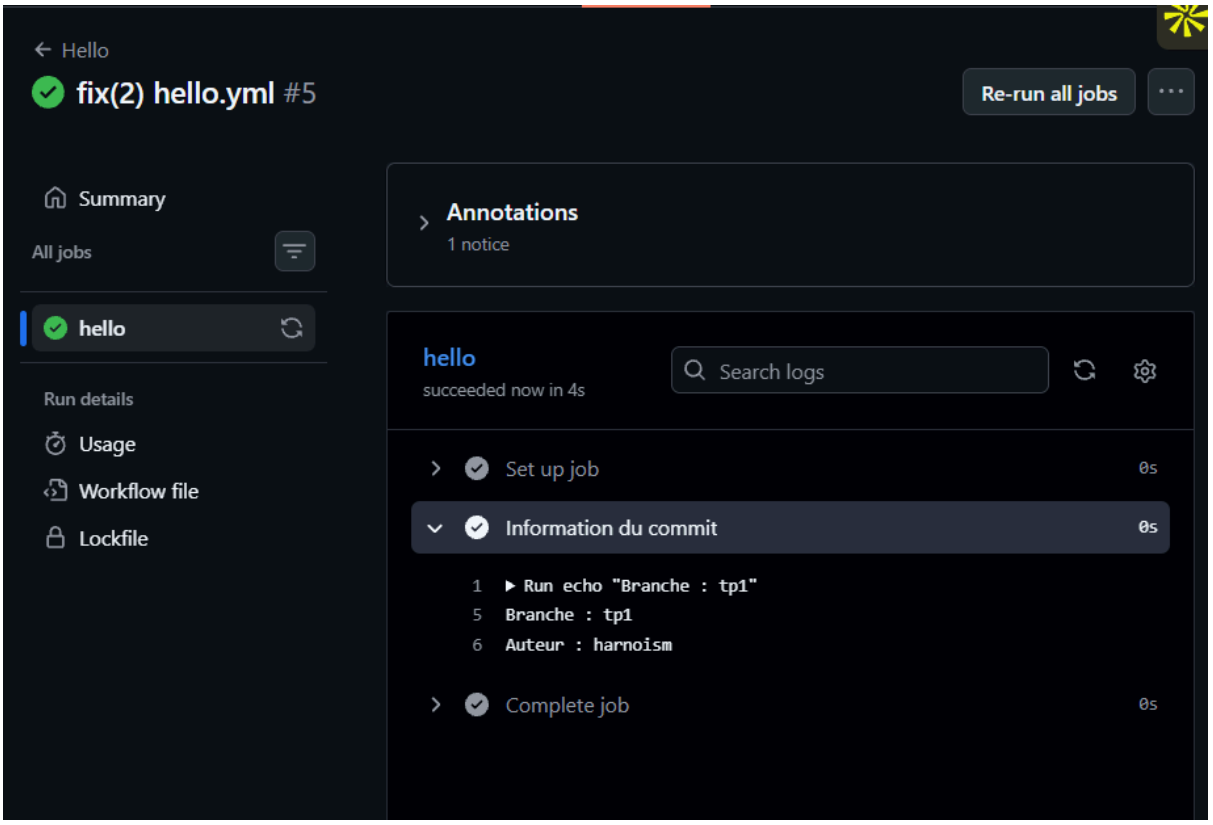
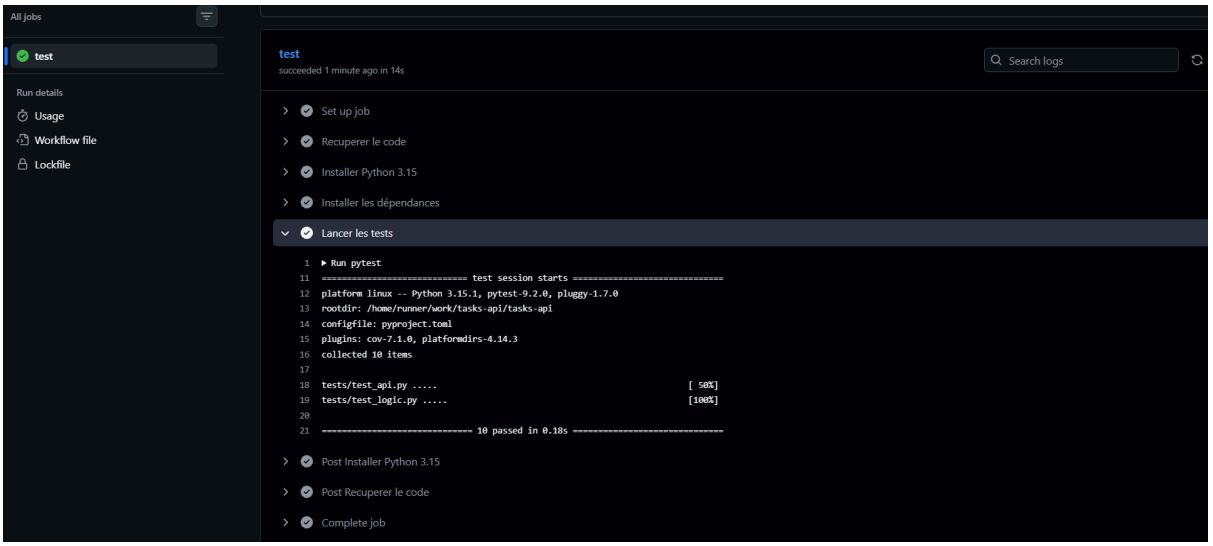
TP1:

TP1-1

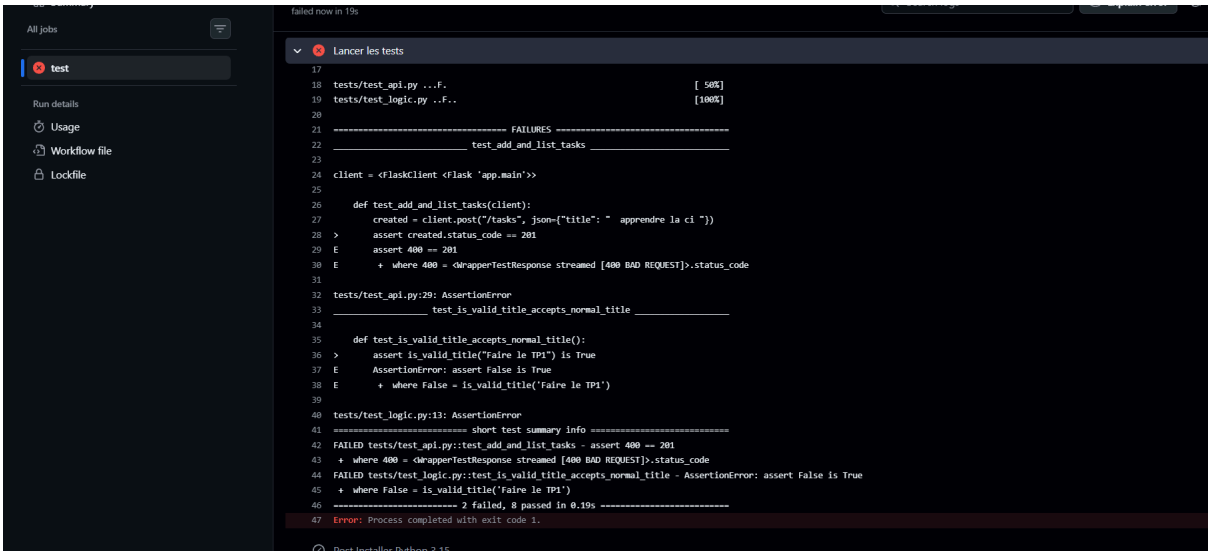


- Combien de temps a duré le run ?
  Le run a duré 4 secondes.
- Où voit-on les logs de chaque étape ?
  On voit les logs au centre de l'écran en dépliant l'étape souhaitée

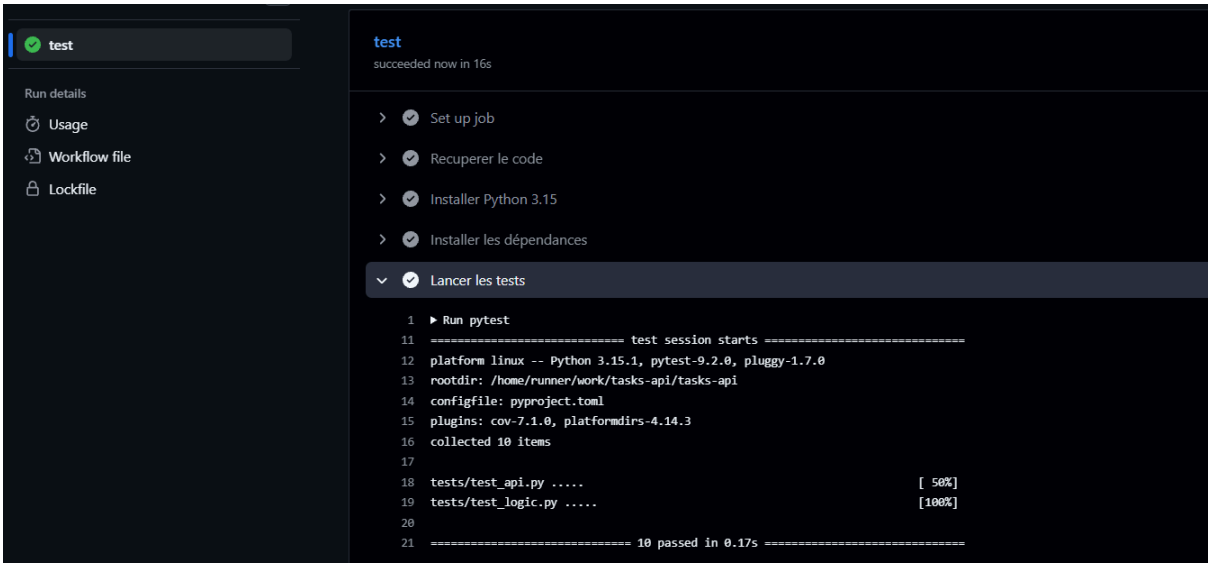
TP1-2



TP1-3

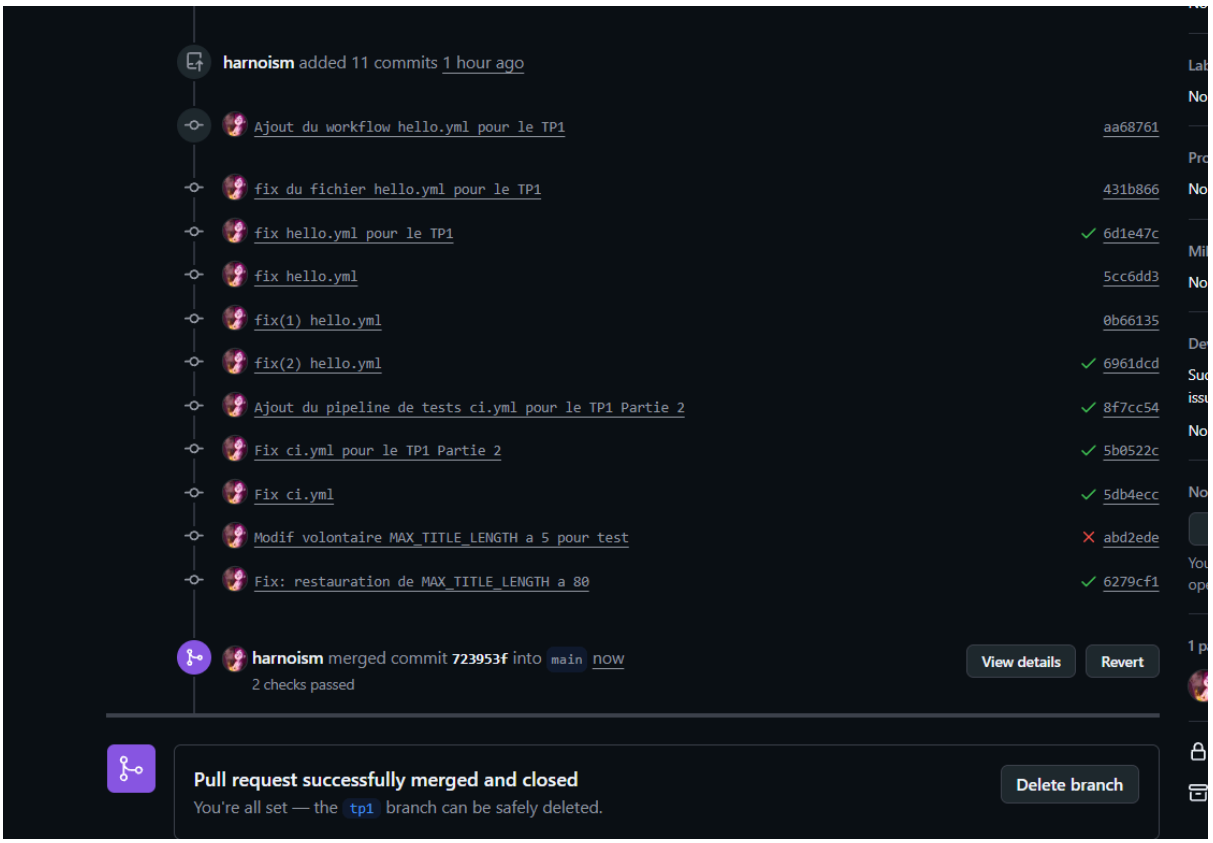


- Quel test a échoué ?
  2 tests ont échoué (comme indiqué à la ligne 46 : 2 failed, 8 passed): - - -
  -test_add_and_list_tasks (dans le fichier tests/test_api.py)
  -test_is_valid_title_accepts_normal_title (dans le fichier tests/test_logic.py)
- À quelle ligne ? L'échec de la logique se produit à la ligne 13 du fichier tests/test_logic.py (indiqué à la ligne 40), L'échec de l'API se produit à la ligne 29 du fichier tests/test_api.py (indiqué à la ligne 32).
- Qu'est-ce qui a permis de le trouver rapidement ?
  C'est le rapport de crash généré par pytest dans les logs. Il affiche clairement une section FAILURES avec : Le chemin complet des fichiers en faute ainsi que leurs numéros de lignes précis (:13 et :29). (Where False = is_valid_title('Faire le TP1')), montrant que le titre "Faire le TP1" (qui fait 12 caractères) renvoie maintenant False au lieu de True car la taille maximale a été abaissée à 5.



TP1-4


harnoism added 11 commits 1 hour ago
Ajout du workflow hello.yml pour le TP1 aa68761
fix du fichier hello.yml pour le TP1 431b866
fix hello.yml pour le TP1 6d1e47c
fix hello.yml 5cc6dd3
fix(1) hello.yml 0b66135
fix(2) hello.yml 6961dcd
Ajout du pipeline de tests ci.yml pour le TP1 Partie 2 8f7cc54
Fix ci.yml pour le TP1 Partie 2 5b0522c
Fix ci.yml 5db4ecc
Modif volontaire MAX_TITLE_LENGTH a 5 pour test abd2ede
Fix: restauration de MAX_TITLE_LENGTH a 80 6279cf1
All checks have passed
2 successful checks
No conflicts with base branch
Merging can be performed automatically.
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully
issues.
None yet
Notifications
1 participant

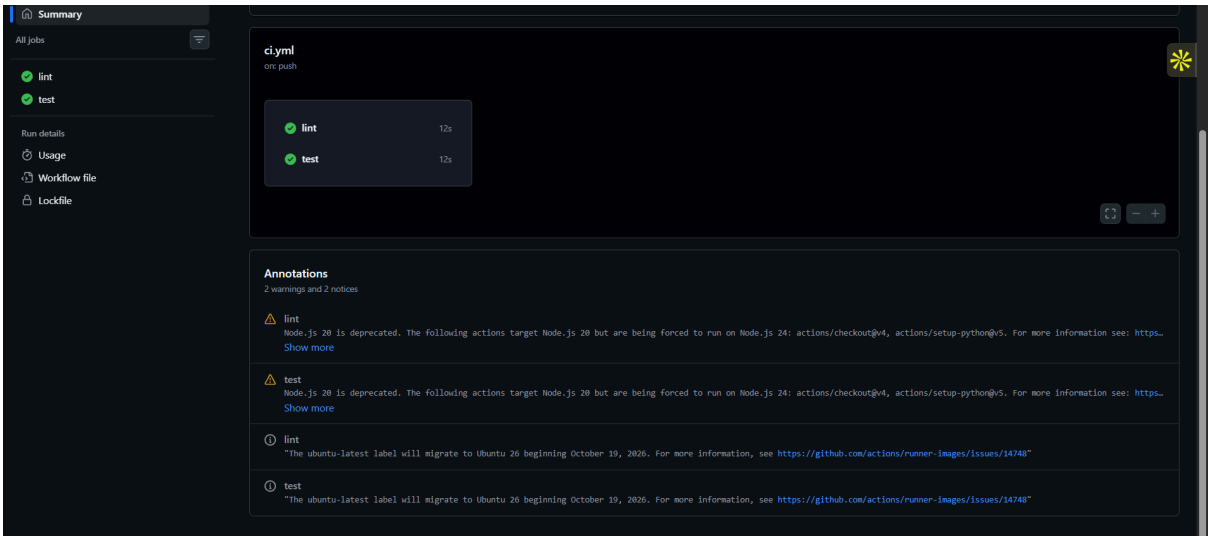

harnoism added 11 commits 1 hour ago
Ajout du workflow hello.yml pour le TP1 aa68761
fix du fichier hello.yml pour le TP1 431b866
fix hello.yml pour le TP1 6d1e47c
fix hello.yml 5cc6dd3
fix(1) hello.yml 0b66135
fix(2) hello.yml 6961dcd
Ajout du pipeline de tests ci.yml pour le TP1 Partie 2 8f7cc54
Fix ci.yml pour le TP1 Partie 2 5b0522c
Fix ci.yml 5db4ecc
Modif volontaire MAX_TITLE_LENGTH a 5 pour test abd2ede
Fix: restauration de MAX_TITLE_LENGTH a 80 6279cf1
harnoism merged commit 723953f into main now
2 checks passed
View details
Revert
Pull request successfully merged and closed
You're all set — the tp1 branch can be safely deleted.
Delete branch

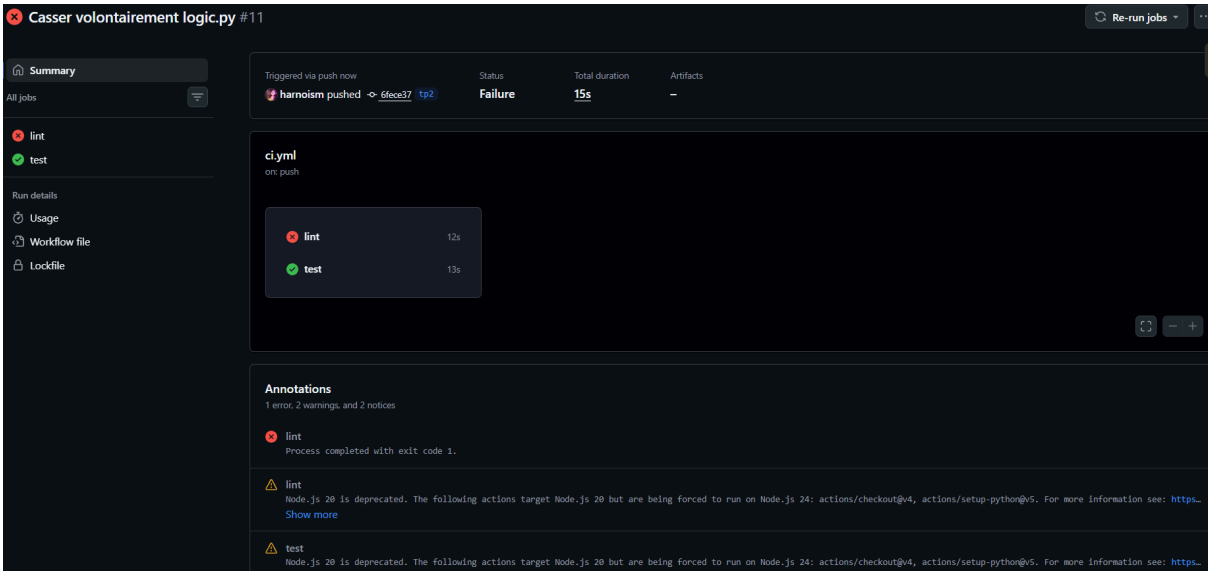


- Pourquoi attend-on que la CI soit verte avant de fusionner ?
  Éviter de casser le projet pour les autres et assurer la qualité du code en continu

TP2

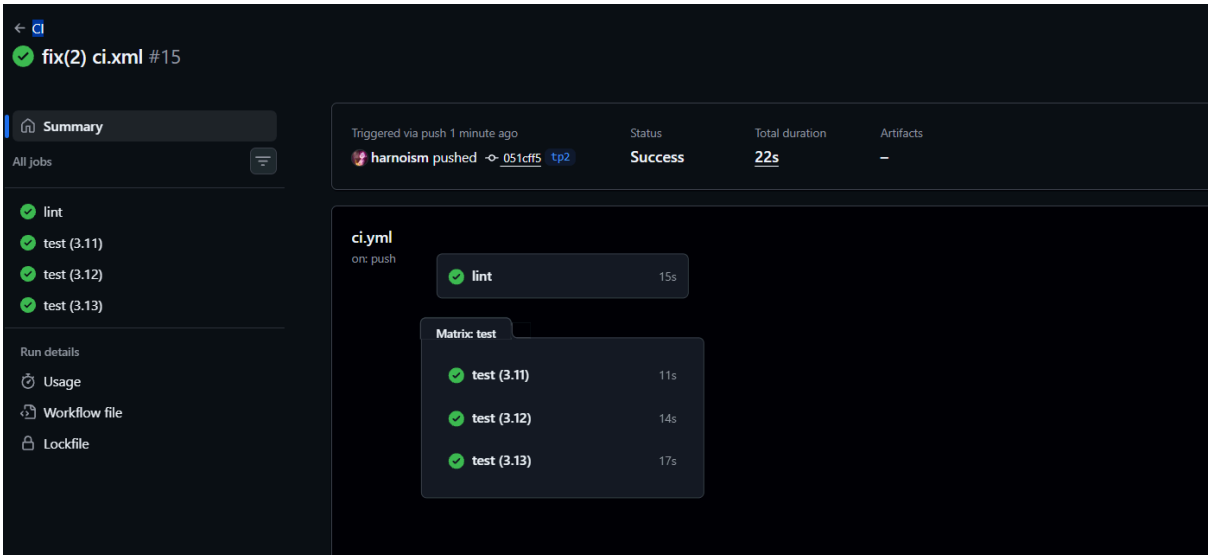
TP2-1

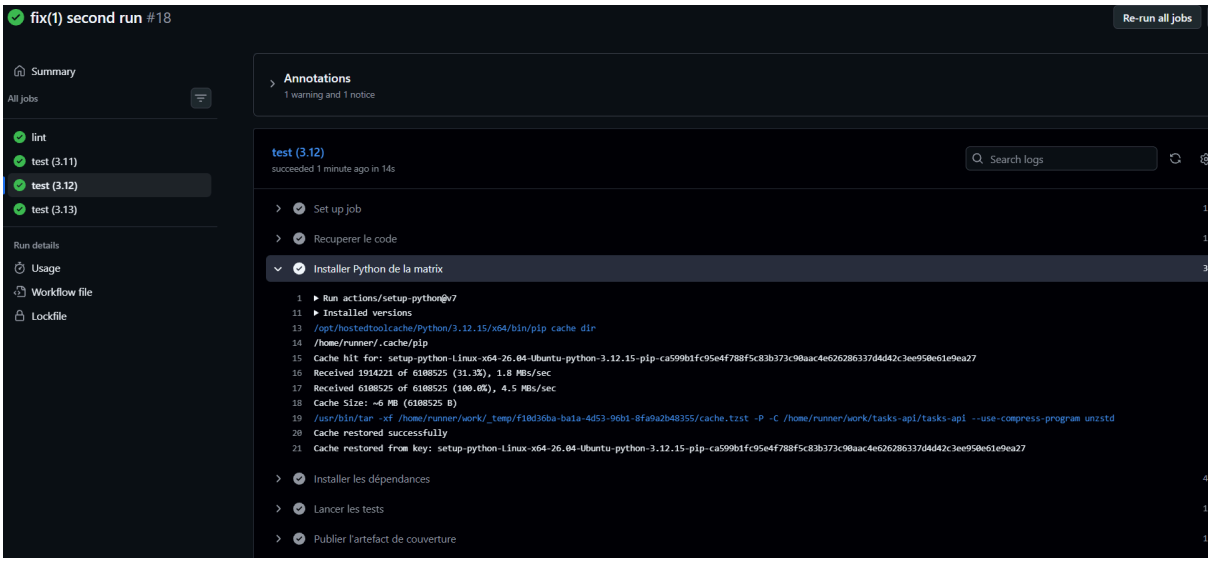


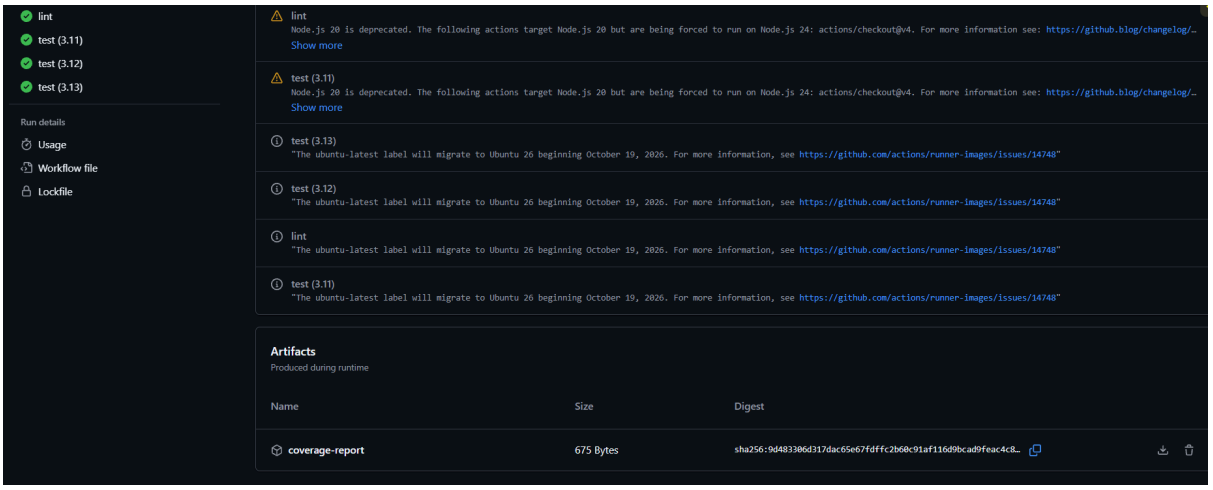


- Quelle différence entre un linter (flake8) et un formateur (black) ? Un linter (flake8) analyse le code source pour détecter les erreurs de syntaxe, les bugs, les mauvaises pratiques (comme les variables ou imports inutilisés). Il signale les problèmes mais ne modifie pas le fichier. Un formateur (black) est "sans compromis" et réécrit automatiquement votre code

TP2-2



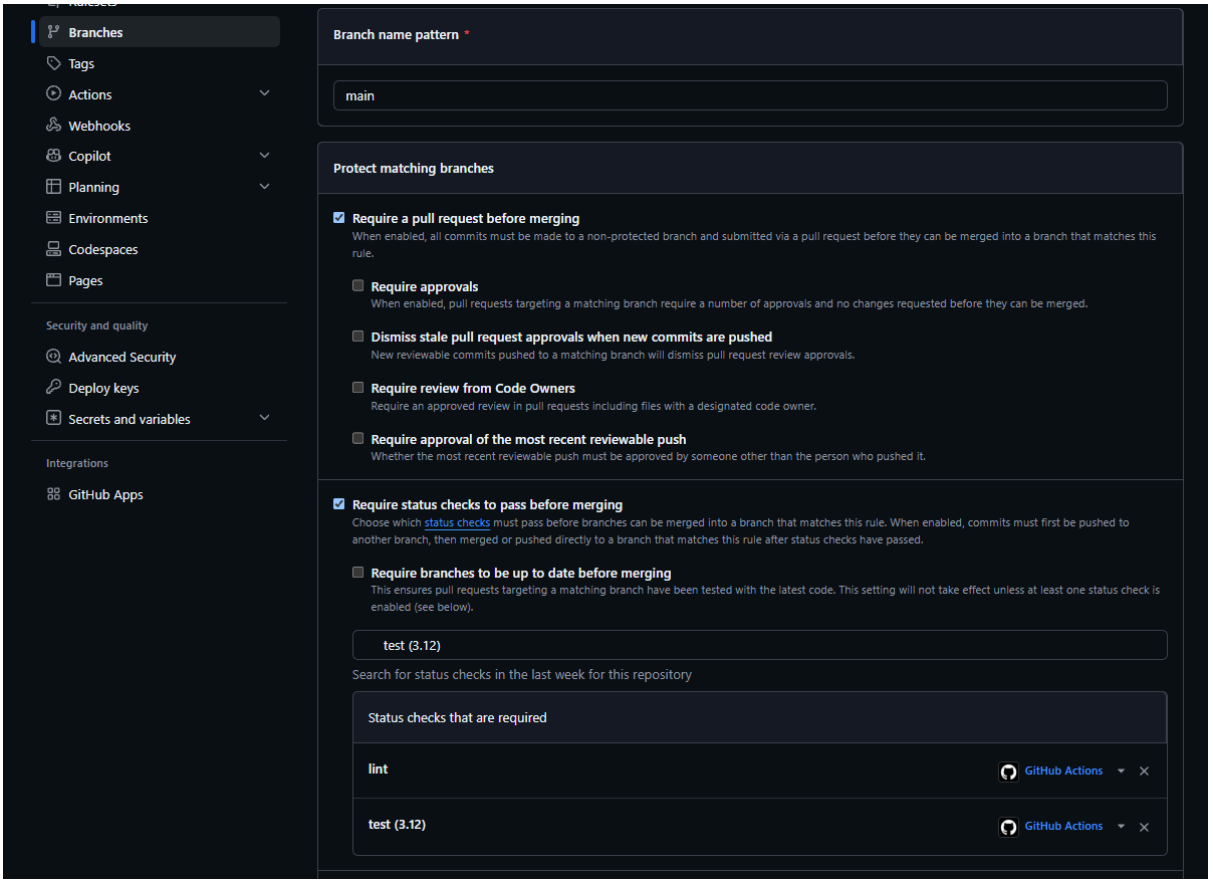




Pourquoi tester sur plusieurs versions de Python ? Tester sur plusieurs versions garantit la rétrocompatibilité de l'application et prévient les régressions en production.

Que gagne-t-on avec le cache ? On gagne un gain de temps et éviter de retélécharger toutes les dépendances à chaque exécutions de la CI
Un artefact, c'est quoi ? Un artefact est un fichier (ou un dossier) généré pendant l'exécution de la CI qu'on décide de sauvegarder pour pouvoir le consulter ou le télécharger plus tard, même après l'extinction de la machine virtuelle.

TP2-3:

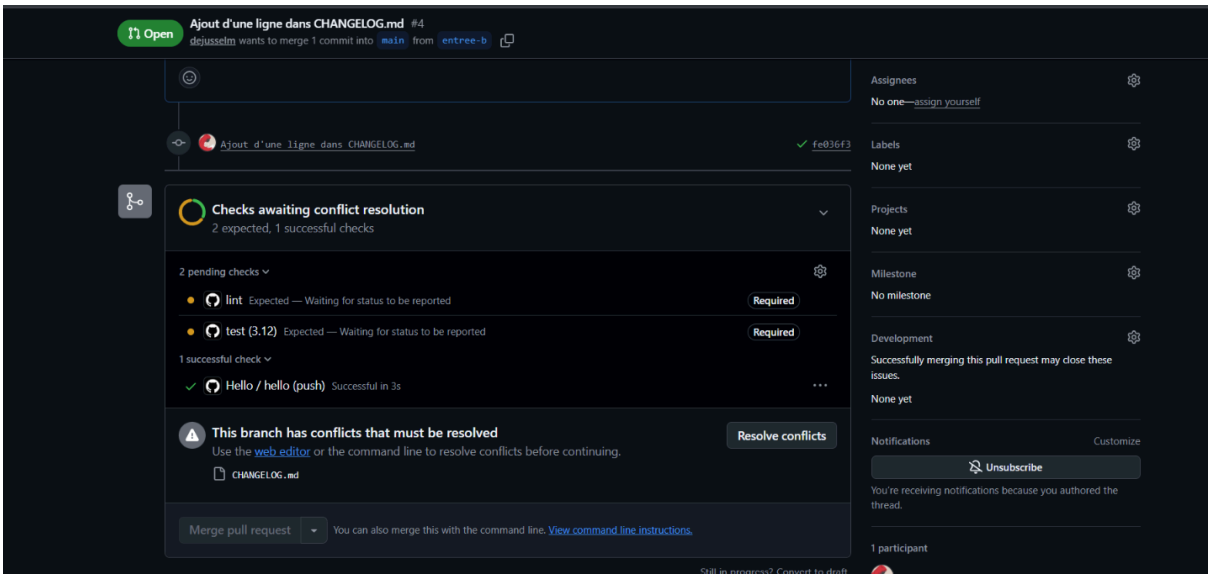


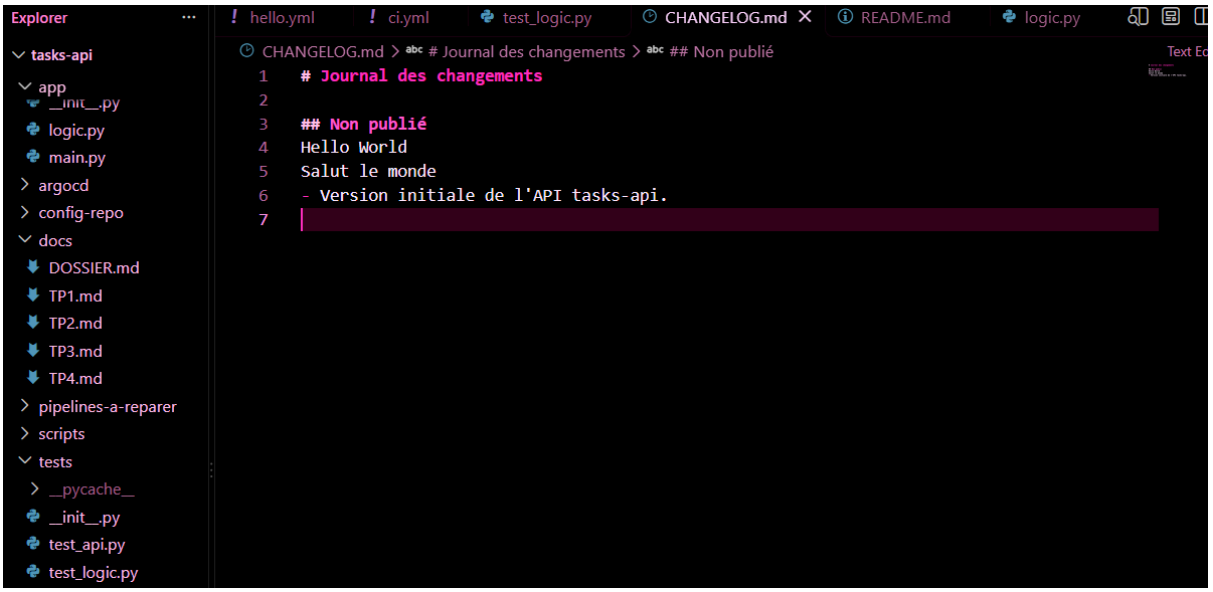


Pourquoi interdire de pousser directement sur le main?

Pour que le main contient en permanence un code stable, propre et prêt à être déployé en production. Cela oblige d'ouvrir une Pull Request, forçant l'exécution des vérifications automatiques (CI) et permettant une revue de code.

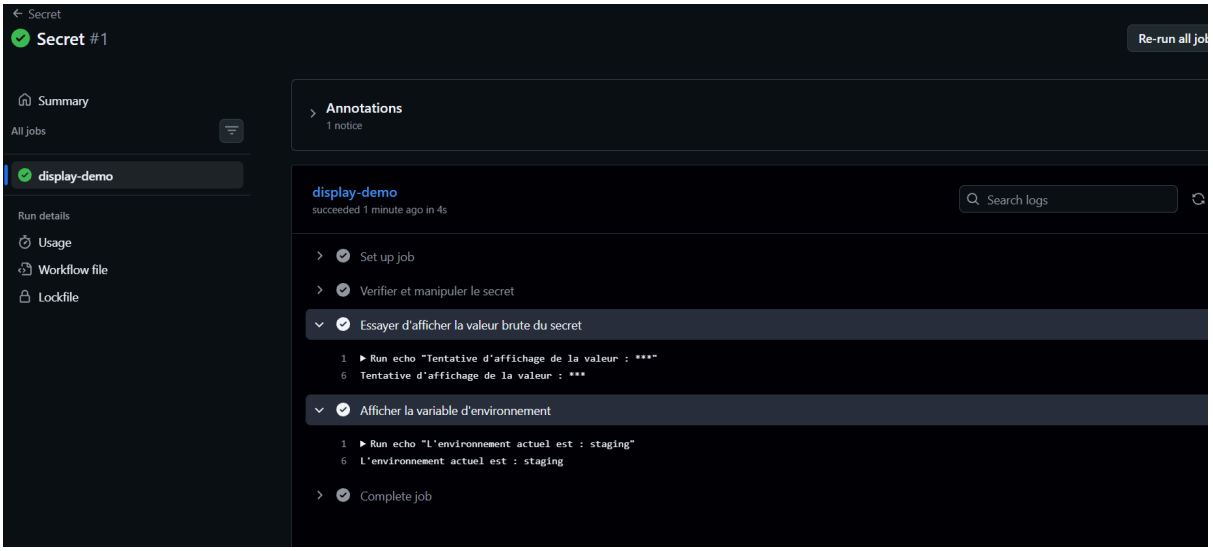
TP2-4:





Pourquoi Git ne peut-il pas résoudre ce conflit seul ?
Git ne peut pas deviner l'intention des développeurs
Qu'est-ce qui a vérifié le code après la résolution ? C'est la pipeline github Action

TP2-5



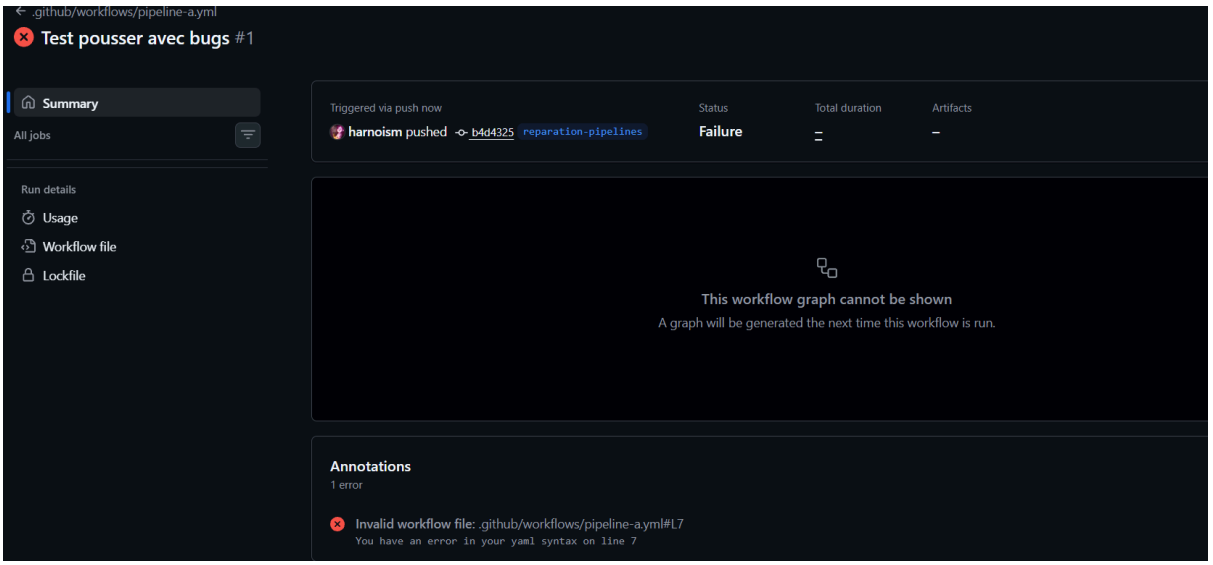
Pourquoi GitHub masque-t-il le secret ? Il le remplace par des astérisque pour éviter la fuite des données non involontaire à la publication
Que se passe-t-il si on le commit dans le dépôt ? Si un secret est écrit directement dans le code et poussé sur GitHub, il est directement corrompu car dans github il y a des bot qui scannent en permanence les commits
Secret ou variable : comment choisir ? Secret sert à chiffrer et masqué les données comme un mot de passe ou un token et une variable, à utiliser les paramètres de configuration globale de l'application ou du pipeline

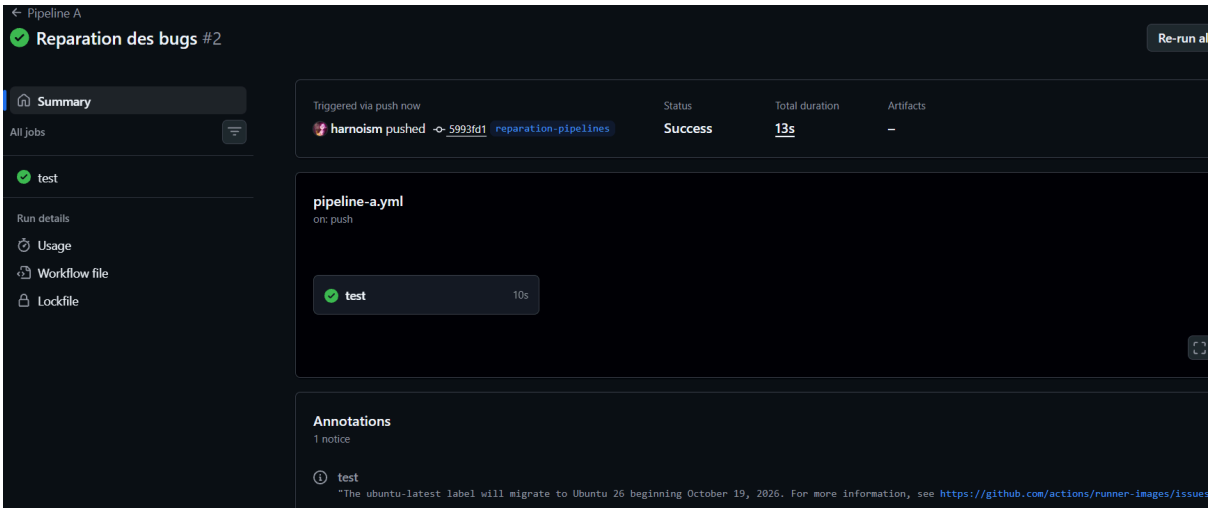
TP2-6
PIPELINE A:

Le Bug : Une erreur d'indentation (un espace en trop) à la ligne - run: pip install -r requirements-dev.txt.

Comment l'avez-vous trouvé :
Au premier push, GitHub Actions refuse de lancer le workflow et affiche un message d'erreur de parsing syntaxique (YAML linting error) pointant la ligne du pip install.

Comment l'éviter à l'avenir : Utiliser les extensions de formatage automatique dans VS Code pour aligner les blocs instantanément.



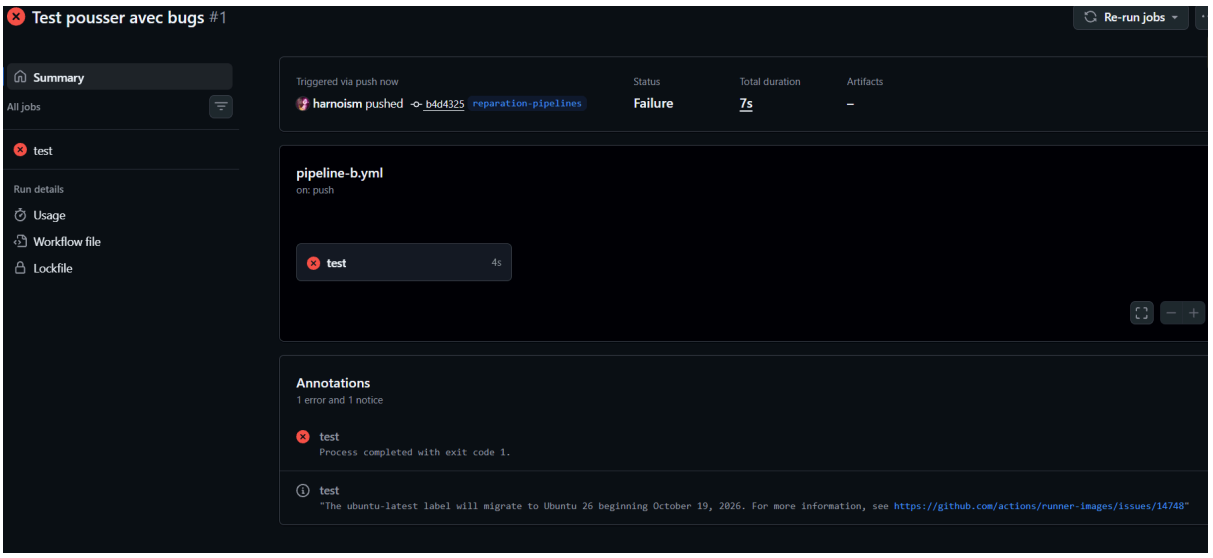


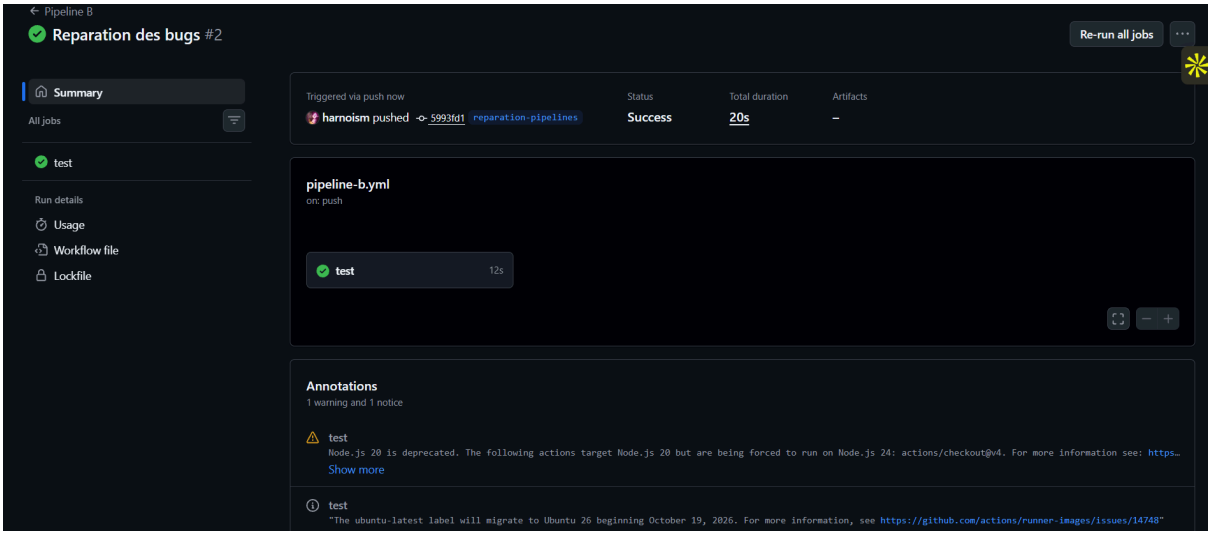
PIPELINE B:

Le Bug : L'oubli de l'action de récupération du code source (actions/checkout). Sans cette étape au tout début des steps, la machine virtuelle de GitHub Actions reste totalement vide. Elle ne contient aucun fichier. L'étape pip install va échouer immédiatement en signalant que le fichier requirements-dev.txt est introuvable.

Comment l'avez-vous trouvé : Dans les logs du run en échec, l'erreur apparaît explicitement lors de l'exécution de la commande de dépendances

Comment l'éviter à l'avenir : Un runner GitHub Actions démarre toujours dans un environnement vierge. Tout job à besoin de l'étape - uses: actions/checkout@v4 en premier.



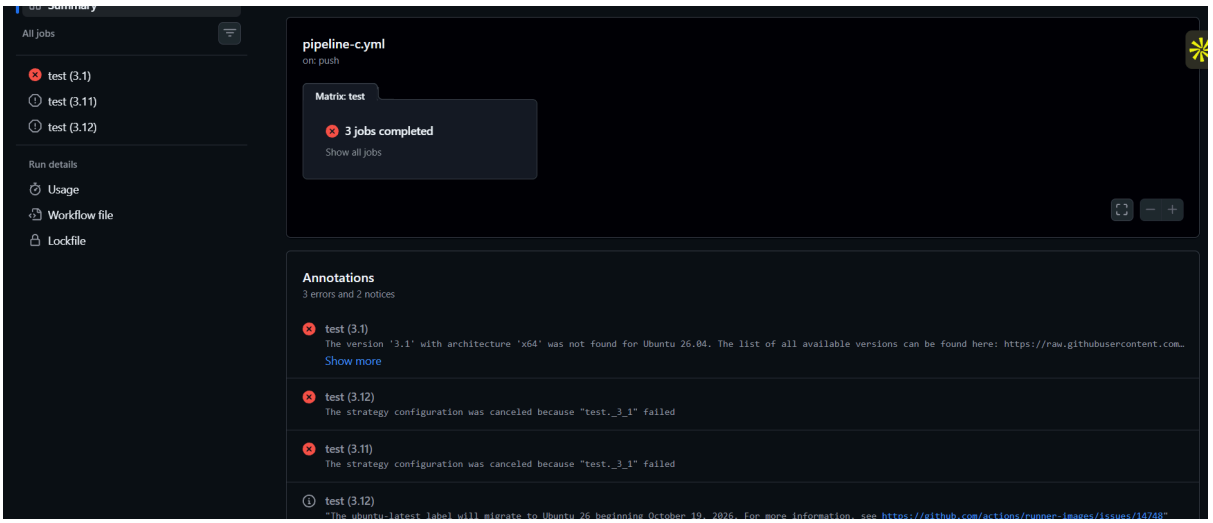


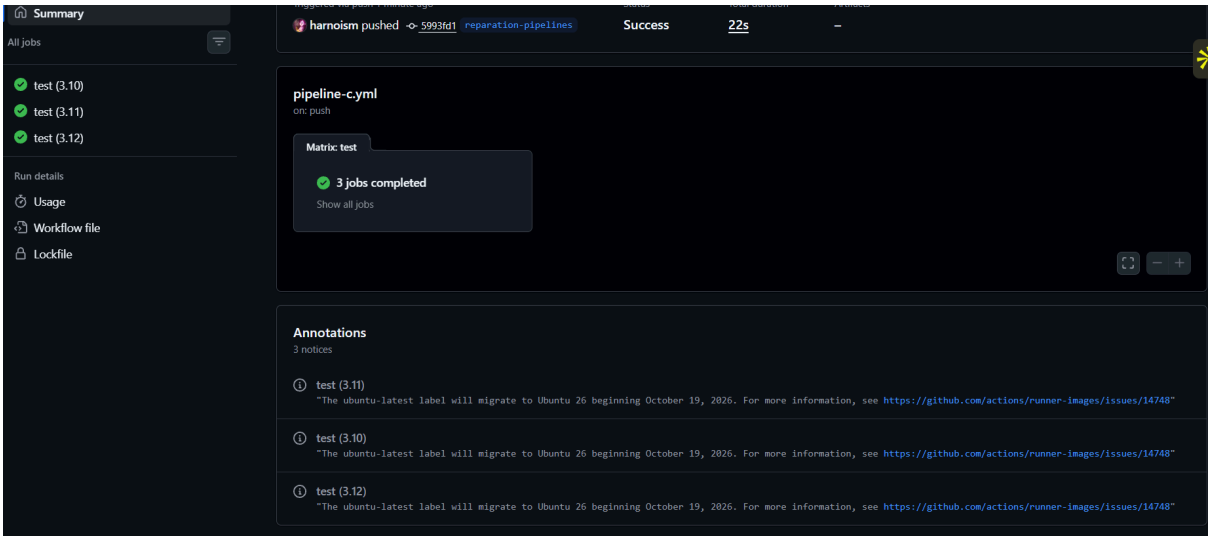
PIPELINE C:

Le Bug : L'absence de guillemets autour des versions de Python.

Comment l'avez-vous trouvé : Les jobs de la matrice s'exécutent mais l'étape setup-python passe au rouge en indiquant qu'il est impossible de trouver ou de télécharger la version de Python demandée.

Comment l'éviter à l'avenir : Entouré systématiquement toutes les chaînes de version dans les fichiers de configuration pour forcer le type String.

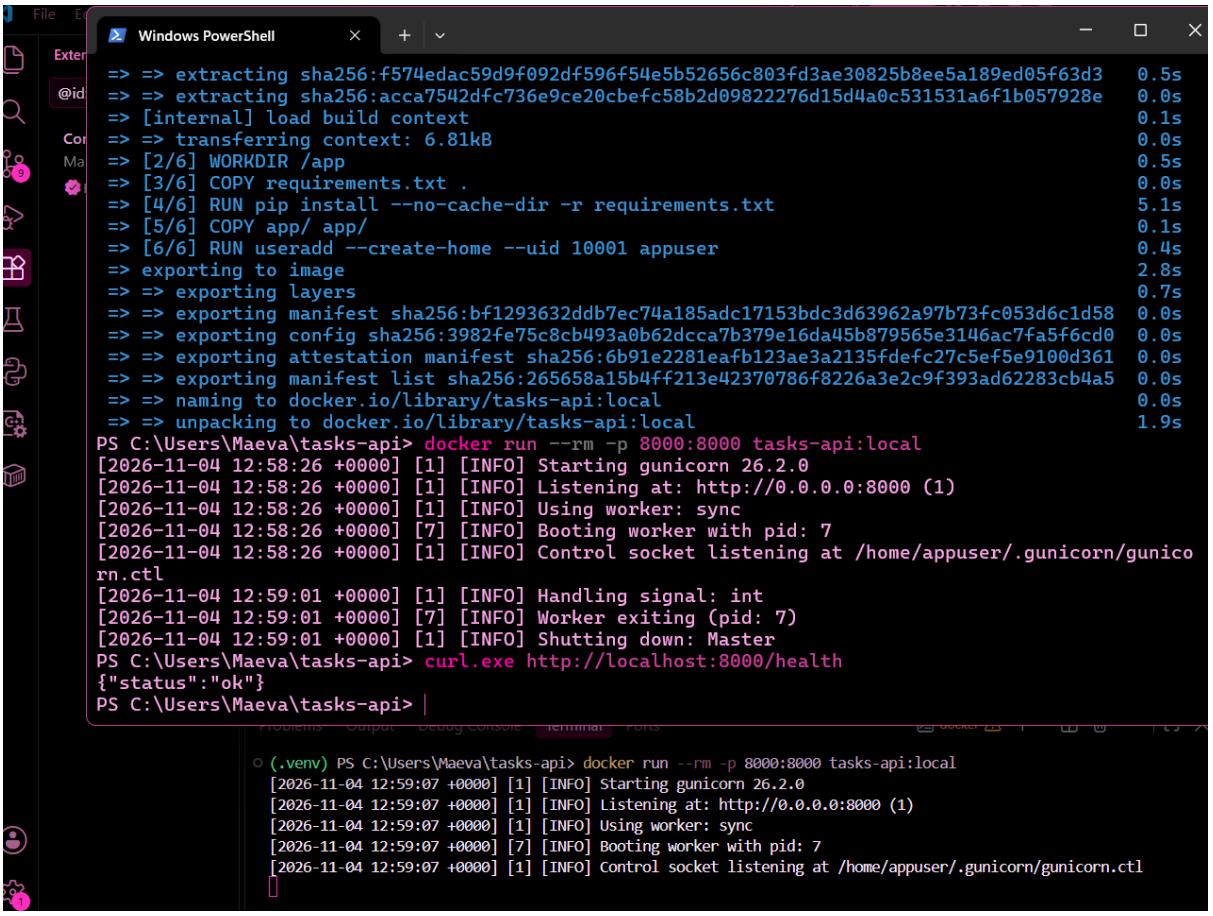

All jobs
test (3.1)
test (3.11)
test (3.12)
Run details
Usage
Workflow file
Lockfile
pipeline-c.yml
on: push
Matrix: test
3 jobs completed
Show all jobs
Annotations
3 errors and 2 notices
test (3.1)
The version '3.1' with architecture 'x64' was not found for Ubuntu 26.04. The list of all available versions can be found here: https://raw.githubusercontent.com…
Show more
test (3.12)
The strategy configuration was canceled because "test._3_1" failed
test (3.11)
The strategy configuration was canceled because "test._3_1" failed
test (3.12)


Summary
All jobs
test (3.10)
test (3.11)
test (3.12)
Run details
Usage
Workflow file
Lockfile
harnoism pushed
5993fd1
reparation-pipelines
Success
22s
pipeline-c.yml
on: push
Matrix: test
3 jobs completed
Show all jobs
Annotations
3 notices
test (3.11)
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
test (3.10)
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
test (3.12)
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"

TP3

TP3-1

```
PS C:\Users\Maeva\tasks-api> docker build -t tasks-api:local .
ERROR: failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the pat
daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\Maeva\tasks-api> docker build -t tasks-api:local .
[+] Building 38.7s (12/12) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                  0.1s
 => => transferring dockerfile: 1.48kB                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                   3.1s
 => [auth] library/python:pull token for registry-1.docker.io                         0.0s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 196B                                                     0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded  26.4s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded8  0.0s
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e 251B / 251B  0.2s
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed 12.12MB / 12.12MB  12.7s
 => => sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed4 1.29MB / 1.29MB  2.3s
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e 29.86MB / 29.86MB  24.7s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5   0.8s
 => => extracting sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35   0.1s
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3   0.5s
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e   0.0s
 => [internal] load build context                                                     0.1s
 => => transferring context: 6.81kB                                                   0.0s
 => [2/6] WORKDIR /app                                                                0.5s
 => [3/6] COPY requirements.txt .                                                     0.0s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                          5.1s
 => [5/6] COPY app/ app/                                                              0.1s
 => [6/6] RUN useradd --create-home --uid 10001 appuser                               0.4s
 => exporting to image                                                                2.8s
 => => exporting layers                                                               0.7s
 => => exporting manifest sha256:bf1293632ddb7ec74a185adc17153bdc3d63962a97b73fc053d6c1d58  0.0s
 => => exporting config sha256:3982fe75c8cb493a0b62dcca7b379e16da45b879565e3146ac7fa5f6cd0  0.0s
 => => exporting attestation manifest sha256:6b91e2281eafb123ae3a2135fdefc27c5ef5e9100d361  0.0s
 => => exporting manifest list sha256:265658a15b4ff213e42370786f8226a3e2c9f393ad62283cb4a5  0.0s
 => => naming to docker.io/library/tasks-api:local                                    0.0s
 => => unpacking to docker.io/library/tasks-api:local                                 1.9s
```



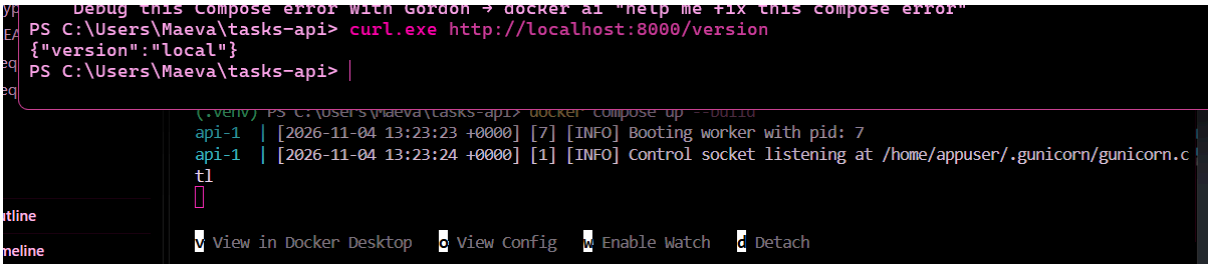
Différence entre une image et un conteneur ?

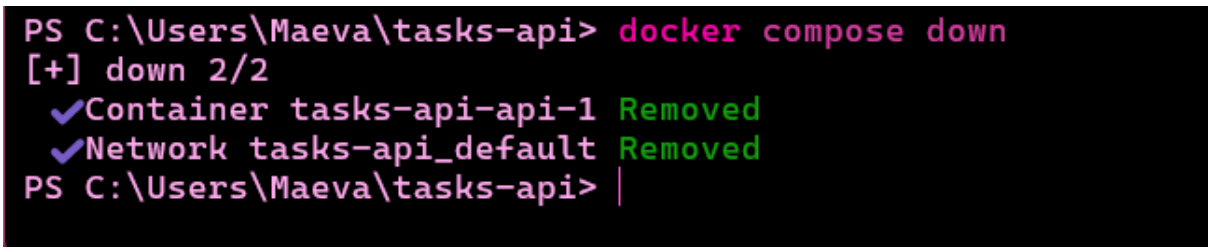
L'image : C'est un modèle figé, en lecture seule, qui sert de patron. Elle contient le système de fichiers minimal. Elle est stockée sur le disque dur et peut être partagée (exemple: Docker-Hub)

Le conteneur : Un processus actif qui tourne en mémoire vive . C'est un simple processus isolé sur la machine hôte qui exécute l'application. On peut lancer plusieurs conteneurs à partir d'une seule et même image. (la cuisine est le repas)

À quoi sert le .dockerignore ?
Le fichier .dockerignore fonctionne comme un .gitignore. Il indique à Docker quels fichiers ou dossiers locaux il doit exclure lors de l'envoi du build.

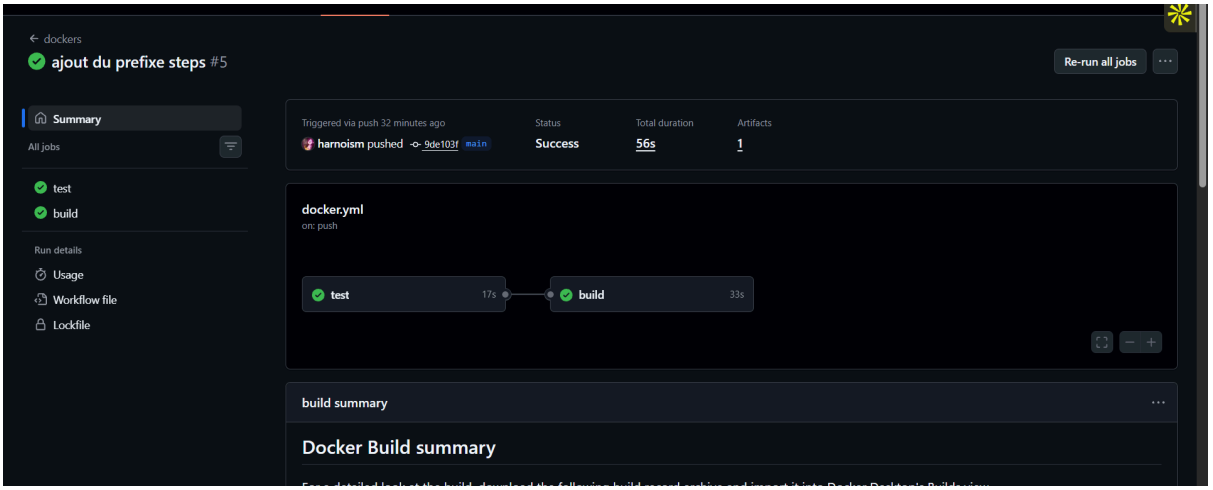
TP3-2

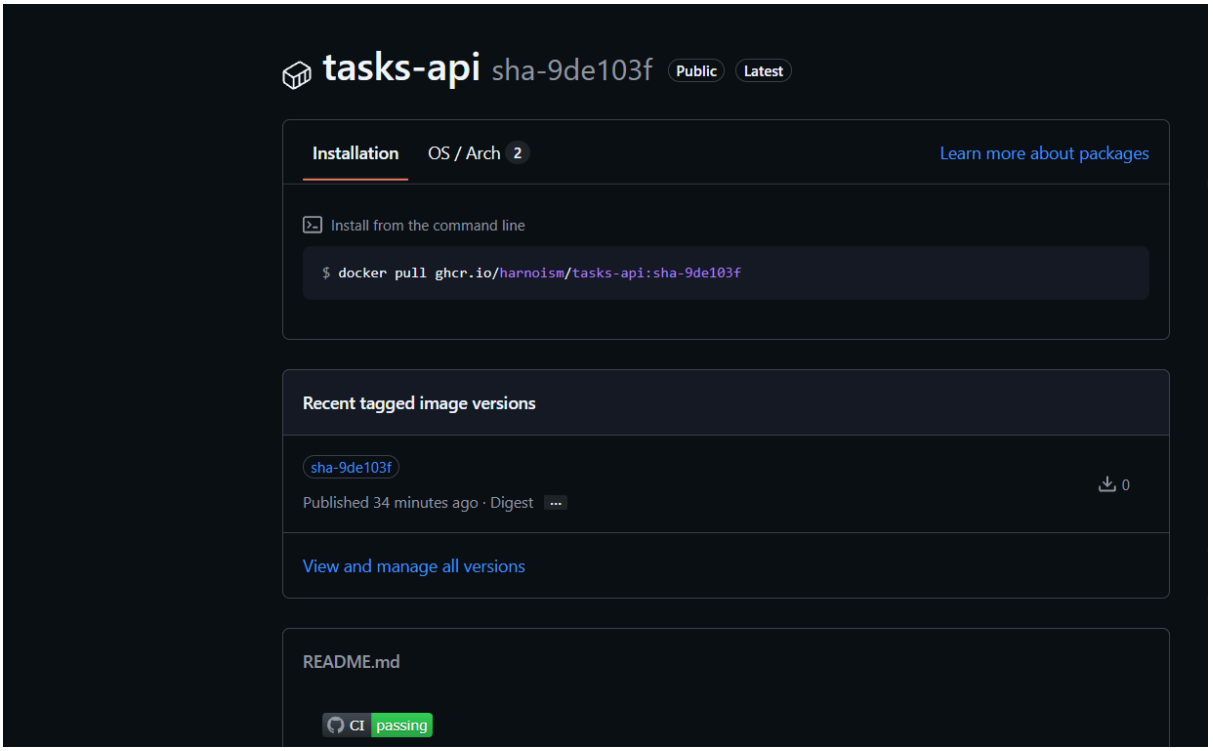




Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ? Simplicité et automatisation, Partage et reproductibilité, Gestion multi-conteneurs (comme lier une api, un cache et une base de données.)

TP3-3





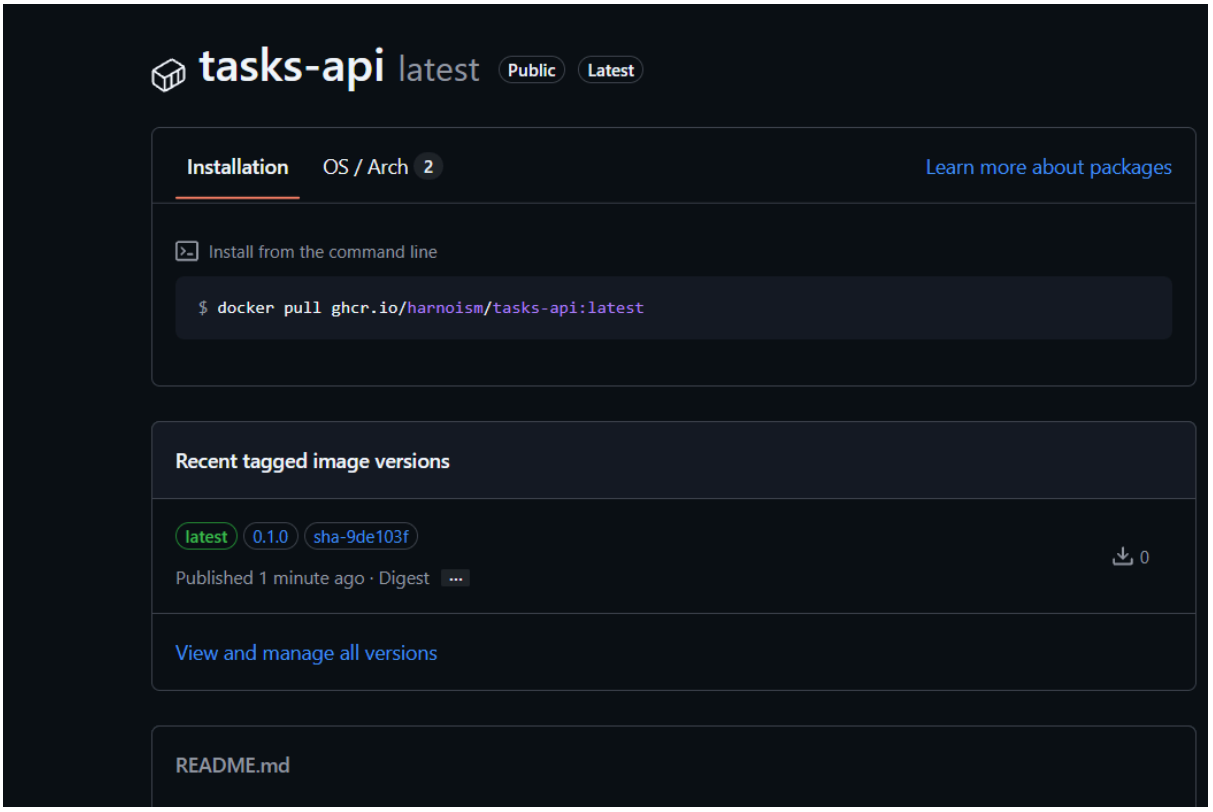
Pourquoi build attend-il test ? Pour garantir qu'aucune image contenant du code défectueux, cassé ou non testé ne soit construite et publiée.

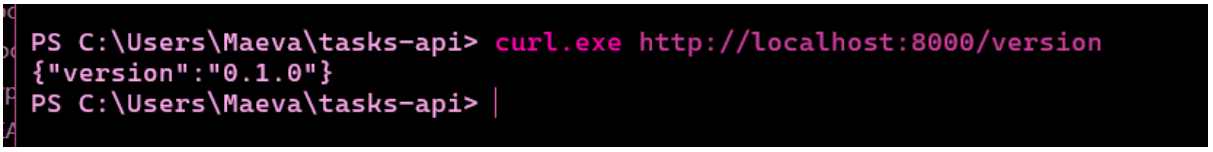
Pourquoi éviter le tag latest en production ?
Il n'est pas stable : Il change à chaque mise à jour.

Il casse la traçabilité : Impossible de savoir précisément quelle version du code tourne.

Il empêche les retours arrière : On ne peut pas facilement revenir à la version précédente

TP3-4

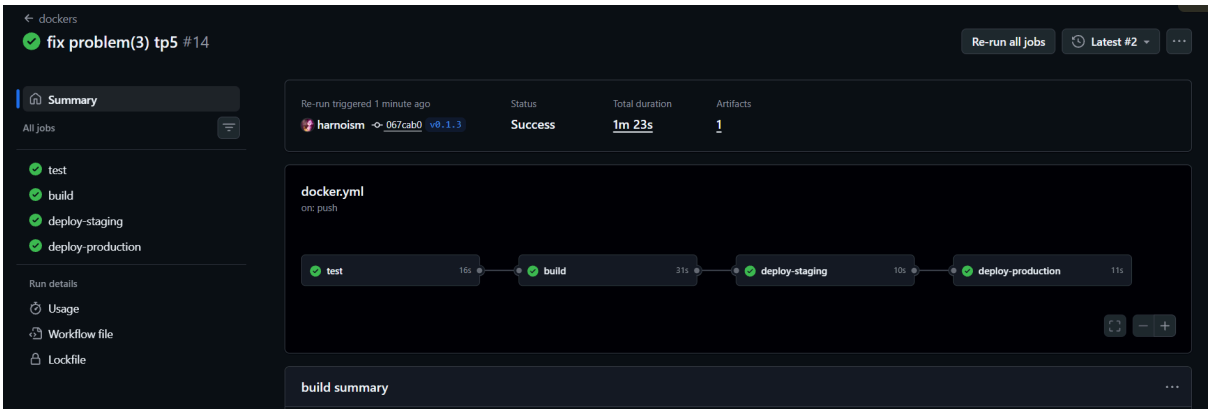




Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

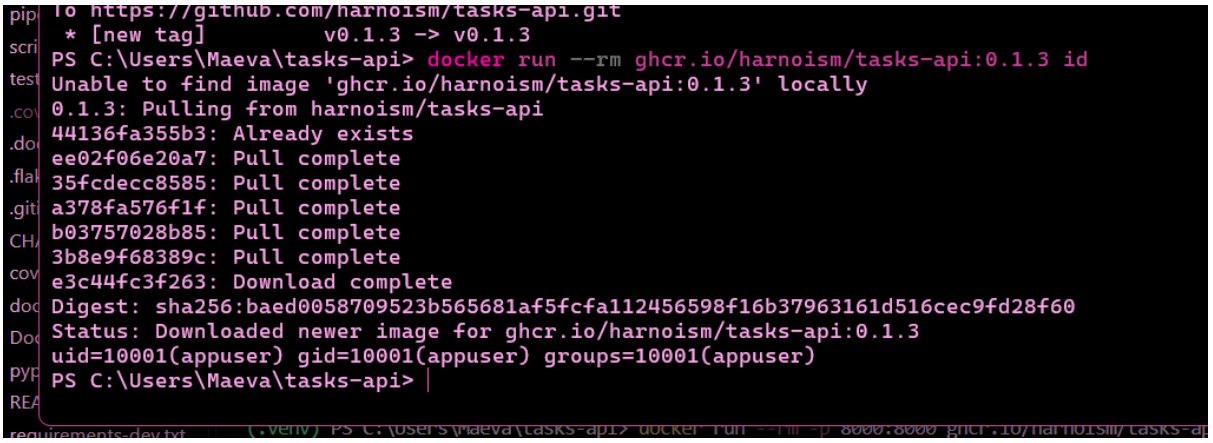
L'avantage d'avoir une image versionnée, c'est surtout pour la reproductibilité. Par exemple, l'image contient déjà le code figé et toutes ses dépendances. Du coup, ça évite complètement le fameux problème du "chez moi ça marche" : l'application va tourner exactement de la même manière sur mon poste, sur le pipeline de CI/CD ou sur le serveur, peu importe la machine.

TP3-5



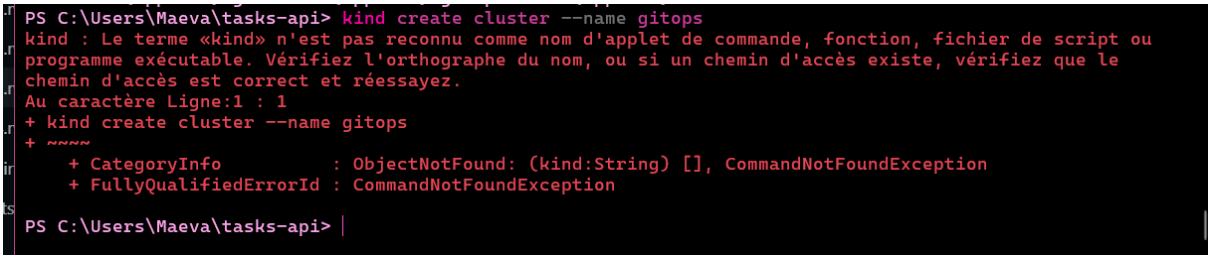
promis monsieur y'avait écrit attente (vu avec mousieur)

TP3-6



Pourquoi ne pas exécuter une application en root dans un conteneur ?

Il ne faut surtout pas lancer une application en root pour des raisons de sécurité, par exemple si un pirate arrive à entrer dans le code il aura tous les droits à cause du root

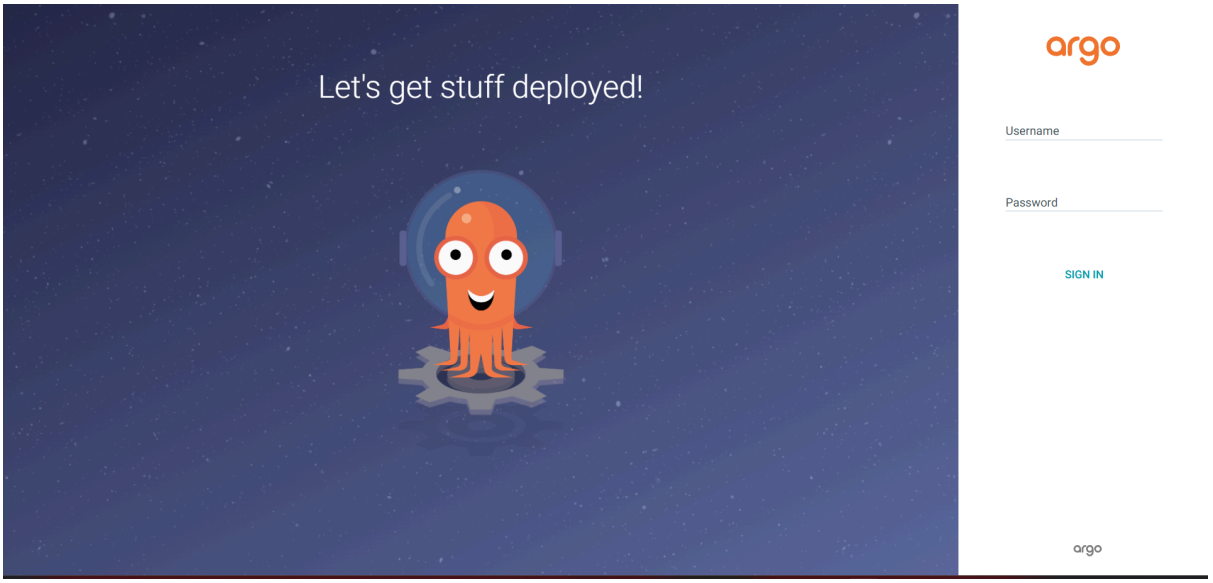


TP4

TP4-1

```
PS C:\Users\Maeva\tasks-api>
PS C:\Users\Maeva\tasks-api> kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   43s   v1.37.0
PS C:\Users\Maeva\tasks-api>
```

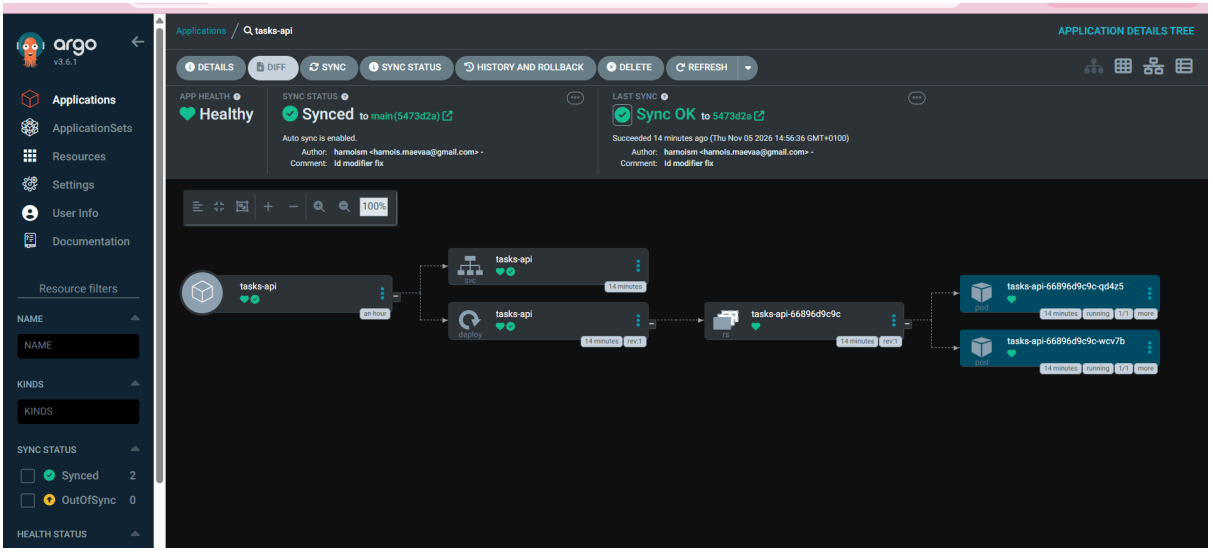
```
PS C:\Users\Maeva\tasks-api> kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0                     1/1     Running   0          4m56s
argocd-applicationset-controller-794bb95c9d-cpp8g   1/1     Running   0          4m56s
argocd-dex-server-66cf64bfcf-n6jd7                  1/1     Running   0          4m56s
argocd-notifications-controller-5878c8d644-4bvp8    1/1     Running   0          4m56s
argocd-redis-5869b7547-hdjn2                        1/1     Running   0          4m56s
argocd-repo-server-7d55cc466b-gs8k9                 1/1     Running   0          4m56s
argocd-server-85b849c688-b2jmv                      1/1     Running   0          4m56s
PS C:\Users\Maeva\tasks-api>
```



C'est quoi un pod ? C’est un ou plusieurs conteneur qui tourne
Un deployment ? C’est un peu comme les Kubernetes, qui va séparer l’application. Ou l’application qui va planté il va le relancer automatiquement (“Un gâteau” ou kubernetes va copier donc refaire 3 fois le même gâteau pour tous les grands mangeurs)
Pourquoi un namespace ? Un espace pour ranger les ressources

TP4-2

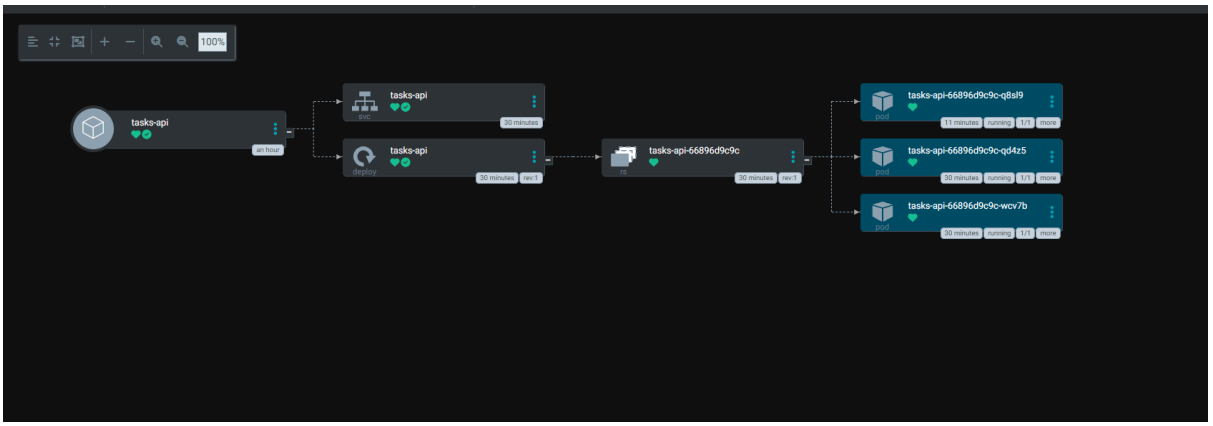
```
>>
(.venv) PS C:\Users\Maeva\tasks-api> kubectl get pods -n tasks -w
NAME                         READY   STATUS    RESTARTS   AGE
tasks-api-66896d9c9c-qd4z5   1/1     Running   0          36s
tasks-api-66896d9c9c-wcv7b   1/1     Running   0          36s
```

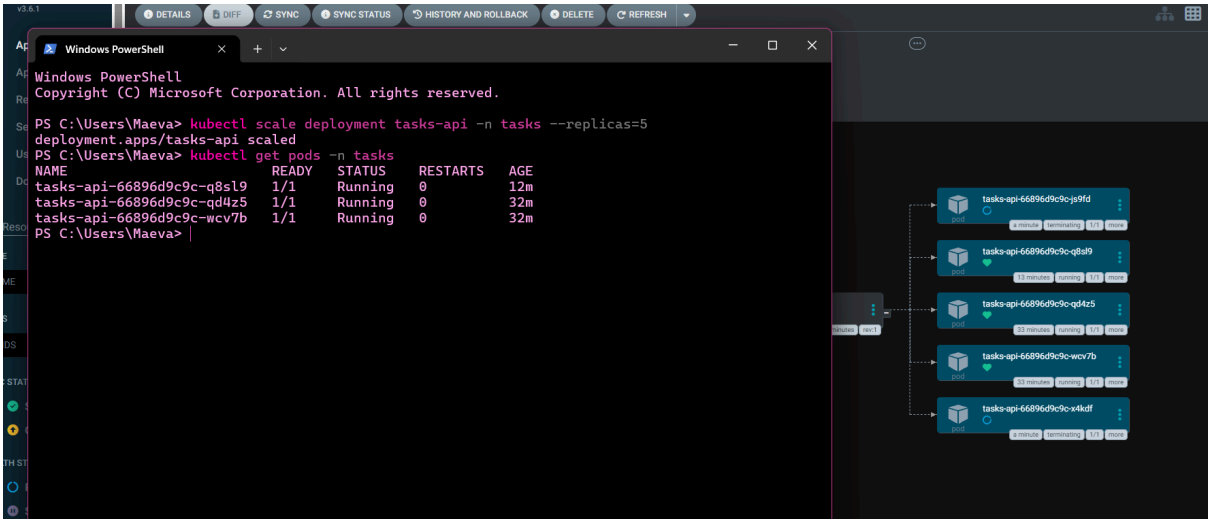




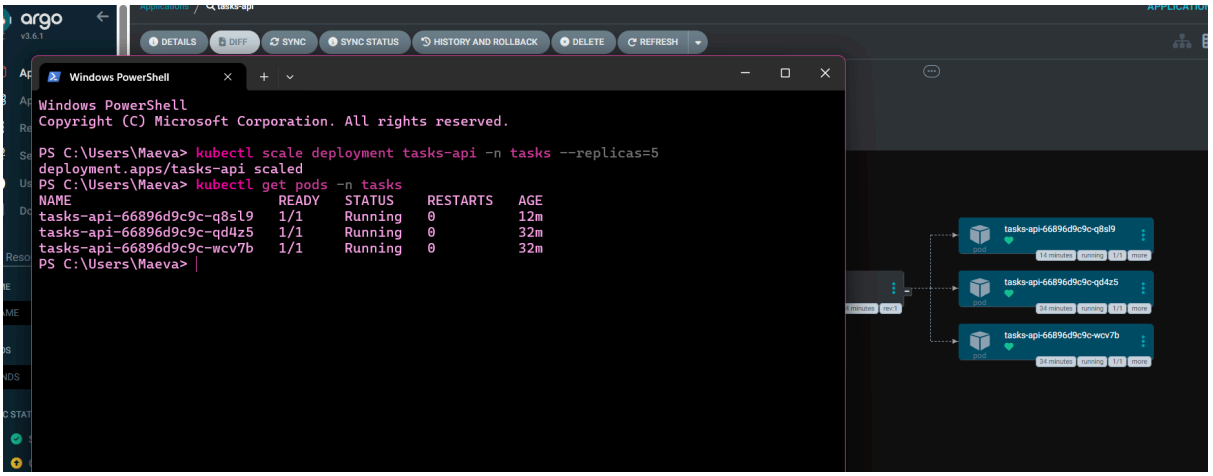
Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ? Pour les sécuriser et bien isoler les pipelines

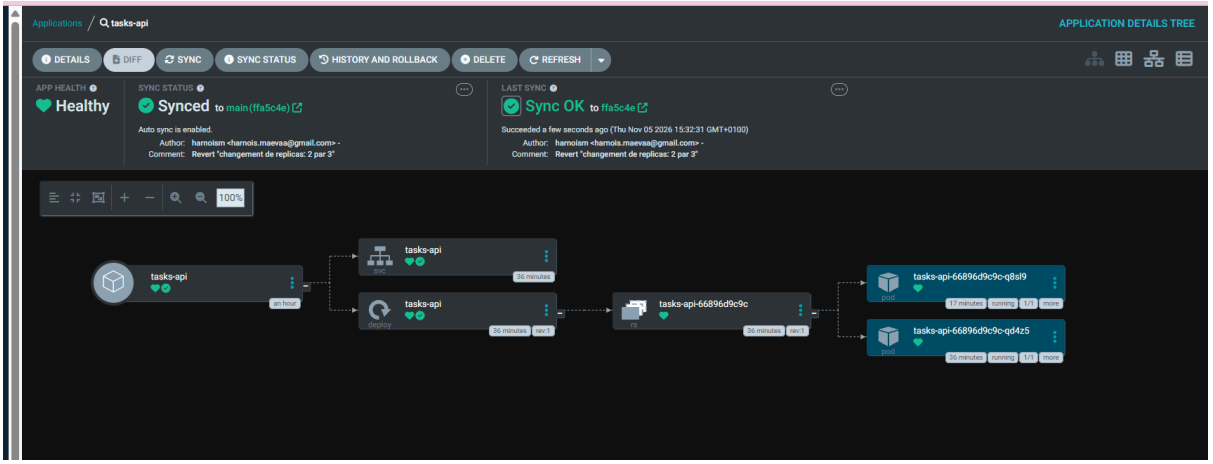
TP4-3

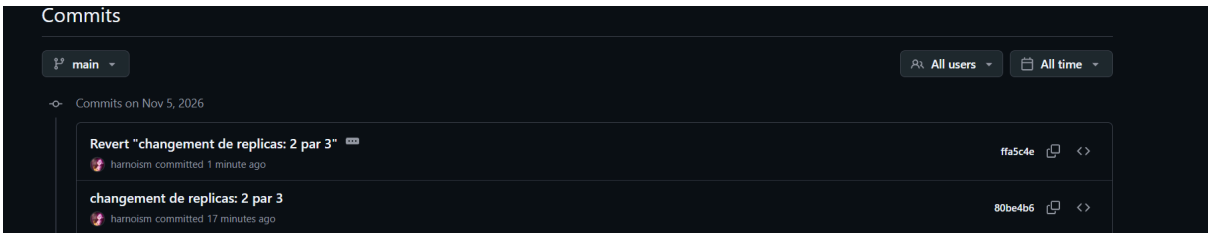




15 secondes après



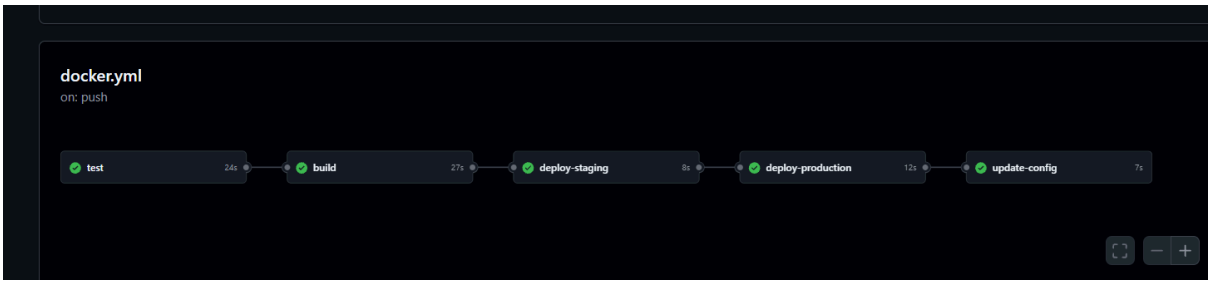


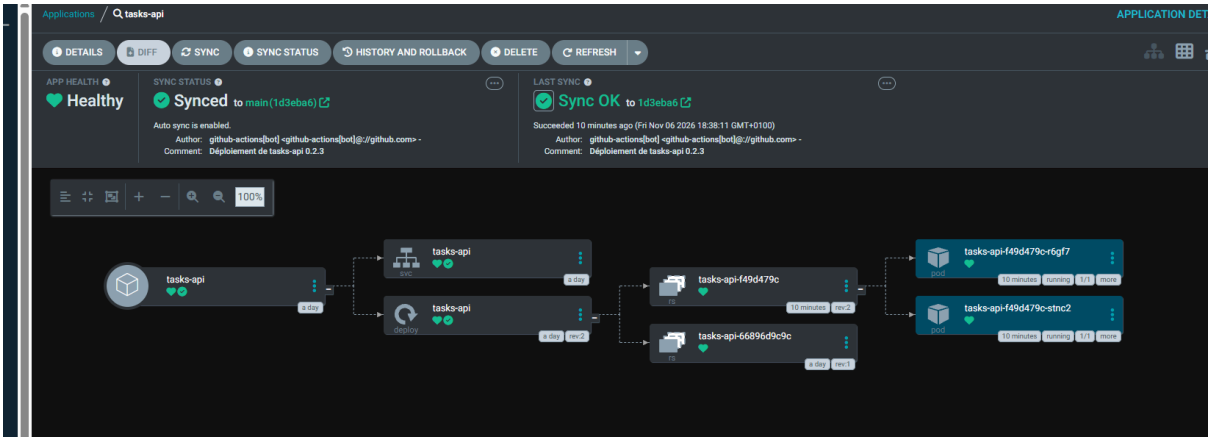


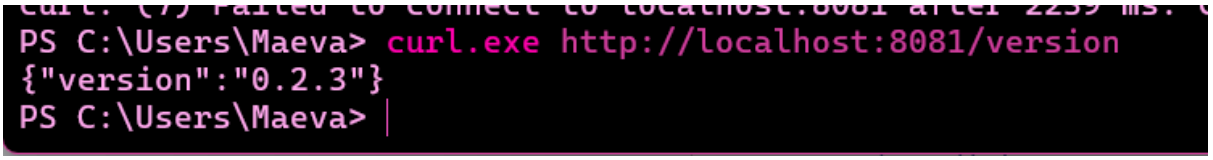
Que signifie selfHeal ? C'est l'auto-guérison automatique qui annule les dérives manuelles sur le cluster

Pourquoi le retour arrière est-il simple en GitOps ? Parce qu'il suffit de faire un git revert pour qu'Argo CD réaligne tout instantanément

TP4-4







Résumez en 5 lignes le chemin d'un git tag jusqu'au pod.

Le développeur pousse le tag v0.2.0, ce qui déclenche le pipeline GitHub Actions.

Le pipeline teste le code, construit l'image Docker et la pousse sur GitHub Packages.

Le job update-config modifie automatiquement le tag de l'image dans le dépôt tasks-config.

Argo CD détecte ce changement sur le dépôt Git de configuration.

Argo CD applique la modification dans le cluster, qui télécharge l'image 0.2.0 et démarre les nouveaux pods

Qui pousse (push) et qui tire (pull)?

Github Action push et Argo CD pull

Quelle différence avec un déploiement CI/CD « classique » ?

la CI possède les clés du cluster et s'y connecte pour forcer le déploiement (Push). En GitOps (Pull), la CI ne touche jamais au cluster : c'est un agent interne (Argo CD) qui surveille Git et met à jour le cluster de manière isolée et sécurisée.